

1-3 · 이미지·컨테이너와 도커 사용하기 · 실습 1.5H

도커 기본 명령 실습

띄우고 · 들여다보고 · 정지하고 · 지우는 한 바퀴

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① **docker version** 으로 클라이언트와 데몬을 구별한다
- ② **nginx:1.27** 로 **run·ps·logs·stop·rm** 한 바퀴를 돈다
- ③ **docker exec** 로 안에 들어가 리눅스임을 확인한다
- ④ 컨테이너 안에서 고친 것이 사라지는 것을 직접 본다

명령을 치기 전에 짧은 이론 셋 — 두 조각 · 두 이름 · 컨테이너의 상태

명령을 치기 전에 ①

도커는 두 조각 — 내가 치는 docker 는 일하는 쪽이 아니다

클라이언트

내가 치는 docker 명령 — 요청을 보낼 뿐 (터미널)

- docker run 은 요청일 뿐
- 식당에서 주문을 받는 사람

데몬 (dockerd)

이미지를 받고 컨테이너를 띄우는 본체 — Docker Desktop / 리눅스 서비스

VS

- 실제로 일하는 쪽 — 요리하는 사람
- 「Cannot connect to the Docker daemon」= 이쪽이 꺼진 것
- Desktop 을 켜거나 `sudo systemctl start docker`

명령 이름이 두 가지로 보이는 이유 — 같은 명령이다

짧은 옛 이름 (이 과정)	정리된 이름	하는 일
<code>docker ps</code>	<code>docker container ls</code>	도는 컨테이너 목록
<code>docker ps -a</code>	<code>docker container ls -a</code>	멈춘 것까지 전부
<code>docker images</code>	<code>docker image ls</code>	받아 둔 이미지 목록
<code>docker rm</code>	<code>docker container rm</code>	컨테이너 지우기
<code>docker rmi</code>	<code>docker image rm</code>	이미지 지우기

「docker 대상 동작」꼴로 정리했는데 옛 짧은 이름도 그대로 두었다 — 이 과정에서는 짧은 쪽을 쓴다

컨테이너에는 상태가 있다 — 명령이 상태를 옮긴다



start → running · stop → exited · rm → 없음 — exited 는 이름과 디스크를 그대로 붙든다

「container name ... is already in use」= 같은 이름이 exited 로 남은 것 → docker rm 이름 (도는 중이면 rm -f)

여섯 단계를 한 바퀴

hello-world 가 Exited 로 남아 있어도 정상 — 6단계에서 한꺼번에 치운다

① 확인

docker version ·
docker run hello-
world

② 받기

docker pull
nginx:1.27 · docker
images

③ 한 바퀴

run -d → ps → curl
→ logs → stop → rm

④ 안으로

docker exec -it
web2 bash → exit

⑤·⑥ 실험·정리

고친 것은 사라진다 →
container prune

nginx 컨테이너로 생명주기 한 바퀴

<code>docker run -d --name web -p 8080:80 nginx:1.27</code>	백그라운드 · 이름 web · 내 8080 → 안 80
<code>docker ps / docker ps -a</code>	도는 것만 / 정지된 것까지
<code>curl http://localhost:8080</code>	「Welcome to nginx!」 HTML — 브라우저로도
<code>docker logs web</code>	로그 보기
<code>docker stop web → docker rm web</code>	정지 → 삭제 — rm 뒤엔 -a 에서도 사라짐

정지된 컨테이너도 이름과 디스크를 차지한다 — 같은 이름으로 다시 run 하면 이름 충돌

컨테이너 안에서 고친 것은 어떻게 되나

1

안에서 페이지를 고친다

`docker exec web2 bash -c 'echo ... > .../index.html'` → curl 에 「내가 고친 페이지」

2

지우고 같은 이미지로 다시

`docker stop web2 && docker rm web2` → 같은 `nginx:1.27` 로 다시 run

3

다시 curl

원래의 「Welcome to nginx!」로 돌아온다 — 이미지는 그대로였고 내 변경은 컨테이너에만 있었다

남길 것은

프로그램은 이미지

데이터는 볼륨

꼬였을 때

원인을 찾지 말고 되돌려서 다시 — 20초면 된다

안 보이면 컨테이너 안 파일부터 확인한다 — 오늘 익힐 디버깅 습관

① 강제로 치우기

```
docker rm -f web web2
```

② 새로 띄우고 고치기

```
run → exec ... echo >  
index.html
```

③ 안 파일부터 확인

```
docker exec web2 cat  
.../index.html
```

④ 그다음 밖에서

```
curl — 옛 페이지면 8080 을  
단 프로그램이 잡은 것
```

단계마다 이것이 보여야 성공

- 1 docker version 에 Server 까지 · hello-world 메시지
- 2 docker images 에 nginx 1.27 · SIZE 약 190~280MB
- 3 curl 에 Welcome to nginx! · stop 뒤엔 ps -a 에만
- 4 안의 /etc/os-release 에 Debian — 안은 리눅스
- 5 고친 페이지가 지우고 다시 만들면 원래대로
- 6 container prune 뒤 docker ps -a 에 아무것도 없다

docker system prune -a 는 이미지까지 지운다 — 지금은 container prune 까지만

1-3 · 실습 1.5H 마무리

다음은 볼륨과 포트로 데이터 다루기 (1H)

PostgreSQL 컨테이너를 볼륨과 함께 띄워, 컨테이너를 지워도 데이터가 남는 것을 직접 확인합니다

✓ `-e · -p · -v` 옵션을 한 명령에 모두 쓴다

✓ heredoc 으로 `psql` 에 SQL 흘려 넣기 (`-i`)

✓ 지웠다 다시 만들어도 두 행이 그대로

✓ 볼륨 없이 만들면 사라지는 것도 확인

지금 해 둘 것

`docker ps -a` 가 비어 있게 정리